Batch 1 — Foundations: Reactive Core and Runtime

> Goal: understand Spring WebFlux from the point of view of a single HTTP request moving through the system.

---

0. The Big Picture First

Before diving into the five topics, fix one mental model:

- Traditional Spring MVC usually follows a thread-per-request model.
- Spring WebFlux follows a small-thread, event-driven, non-blocking model.
- WebFlux does not magically reduce the latency of one slow database call.
- WebFlux reduces the cost of waiting, so the system can survive more concurrent I/O-heavy work with fewer threads.

Normal Servlet Request Flow

1. Request hits Tomcat/Jetty worker thread.
2. Controller runs on that worker thread.
3. If code calls a blocking database or HTTP client, that same thread waits.
4. While waiting, the thread is occupied but doing no useful work.
5. Response is written only after the blocking work finishes.

WebFlux Request Flow

1. Request hits Netty event loop.
2. Handler builds a reactive pipeline and returns `Mono` or `Flux`.
3. If downstream work is non-blocking, the thread registers interest and is freed.
4. When data is ready, the runtime resumes the pipeline with a signal.
5. Response is written when the publisher completes or emits enough data.

The One-Line Core Ideology

WebFlux is about saying: "Do not waste an expensive thread just because we are waiting for I/O."

Where WebFlux Wins

- High concurrency
- Heavy I/O wait time
- Streaming APIs
- Reactive end-to-end stack using non-blocking clients and drivers

Where WebFlux Backfires

- Blocking JDBC everywhere
- CPU-heavy request processing
- Small apps where MVC is already sufficient
- Teams that do not understand reactive debugging and operator semantics

---

1. Reactive Programming Model

What It Changes in the Request Flow

In a normal flow, your code says "do step 1, then block, then do step 2." In WebFlux, your code says "describe what should happen when data arrives, when an error happens, and when the stream completes."

That changes the request from a direct step-by-step procedure into a signal-driven pipeline.

Core Idea

Reactive programming is a model where data moves as signals through a pipeline, and the consumer can control demand.

The key contract is:

- `Publisher` produces signals
- `Subscriber` consumes signals
- `Subscription` links them and carries `request(n)` and `cancel()`

Real-World Analogy

Think of a restaurant with table pagers:

- In the blocking world, one waiter stands beside you the whole time until your table is ready.
- In the reactive world, the host gives you a pager and moves on.
- When your table is ready, the pager buzzes and the next step happens.

The host is not wasting a person just because you are waiting.

Why It Is Better Than Normal Flow

- Waiting becomes cheap.
- Fewer threads can handle more concurrent requests.
- Backpressure is part of the model.
- Streaming becomes natural instead of bolted on.

How It Works

1. A publisher defines how data will be produced.
2. A subscriber subscribes.
3. The publisher calls `onSubscribe` with a `Subscription`.
4. The subscriber asks for data using `request(n)`.
5. The publisher emits `onNext` up to the requested amount.
6. The stream ends with `onComplete`, `onError`, or `cancel`.

The WebFlux-Specific Difference

Spring MVC is centered around immediate values and blocking calls.

Spring WebFlux is centered around `Publisher` values and delayed completion. The framework does not need the final value right now. It only needs a publisher that will eventually emit it.

Backfires That Might Occur

- Reactive code becomes harder to reason about when developers think in hidden blocking terms.
- Stack traces can be harder to read.
- Shared mutable state becomes more dangerous because flow is asynchronous.
- If the whole stack is not reactive, complexity rises fast and benefits collapse.

Anti-Patterns

- Treating reactive code as just "async syntax" without understanding signals.
- Mixing blocking I/O into the middle of the reactive chain.
- Using side effects everywhere instead of pure transformations.
- Building a reactive API on top of a fully blocking architecture and expecting magic.

Interview Punchline

Reactive programming is not mainly about speed. It is about resource efficiency under concurrent I/O wait, signal-driven composition, and built-in demand control.

Code Sample (Java)

```
Flux<Integer> numbers = Flux.range(1, 5);

numbers.subscribe(new BaseSubscriber<>() {
    @Override
    protected void hookOnSubscribe(Subscription subscription) {
        request(2);
    }

    @Override
    protected void hookOnNext(Integer value) {
        System.out.println("Received: " + value);
```

```
            request(1);
        }
    });
```

This sample matters because it shows the real reactive contract: subscription happens first, then the consumer explicitly asks for data.

Interview Trap

"Reactive means the producer just keeps pushing data and the consumer has no control."

That is wrong. In Reactive Streams, demand is part of the contract through `request(n)`.

Quick Revision Notes

- Reactive = signal-driven, not step-by-step blocking execution.
- Main value = better concurrency under I/O wait.
- Core contract = `Publisher`, `Subscriber`, `Subscription`.
- Trap to remember = reactive is not automatically faster for CPU-heavy work.

---

2. Mono and Flux

What They Change in the Request Flow

In Spring MVC, a controller usually returns the final object directly.

In WebFlux, the controller returns a promise-like stream description:

- `Mono<T>` means 0 or 1 item
- `Flux<T>` means 0 to many items

The framework subscribes later and writes the response as the signals arrive.

Simple Analogy

- `Mono` is like ordering one package delivery. It may arrive once, or not at all.
- `Flux` is like subscribing to a newspaper. Many editions may come over time.

Why They Matter

- They encode cardinality directly.
- They allow lazy execution.
- They let Spring defer work until subscription time.
- `Flux` makes streaming and chunked responses natural.

How They Work in a Request

```
@GetMapping("/users/{id}")
Mono<UserDto> getUser(@PathVariable String id) {
    return userService.findById(id);
}

@GetMapping(value = "/events", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
Flux<EventDto> streamEvents() {
    return eventService.stream();
}
```

What actually happens:

1. The controller does not return the user itself.
2. It returns a recipe for obtaining the user.
3. Spring WebFlux subscribes to that publisher.
4. When `Mono` emits, the response body is written.
5. When `Flux` emits multiple items, Spring can stream them progressively.

The Critical Interview Idea: Nothing Happens Until Subscription

This is one of the most important WebFlux truths.

- Operator chain creation is assembly time.
- Real execution starts at subscription time.

That means this code is only a pipeline definition:

```
Mono<String> mono = Mono.just("hello")
    .map(String::toUpperCase)
    .doOnNext(System.out::println);
```

Printing happens only when someone subscribes.

Assembly Time vs Execution Time

This distinction explains many bugs:

- Assembly time: build the pipeline
- Execution time: signals actually flow

This is why `Mono.just(blockingCall())` is dangerous. The blocking call happens immediately during assembly, before subscription.

Why It Is Better Than Normal Flow

- Delayed execution

- Natural composition across async boundaries
- Easy modeling of one result vs many results
- Better fit for streaming HTTP responses and server-sent events

Backfires That Might Occur

- Using `Flux` when you really have one item confuses semantics.
- Using `Mono<List<T>>` everywhere when the API is truly a stream loses reactive value.
- Multiple subscriptions can trigger repeated work if the publisher is cold.
- Confusing `Mono` with `Optional` or `CompletableFuture` leads to shallow usage.

Anti-Patterns

- Calling `subscribe()` manually inside controller or service code.
- Wrapping blocking code in `Mono.just(...)`.
- Returning `Mono` only at the controller and keeping everything underneath blocking.
- Using `block()` in the request path.

Interview Punchline

`Mono` and `Flux` are not containers of data. They are publishers of future signals, and Spring WebFlux subscribes to them to drive the HTTP response.

Code Sample (Java)

```
Mono<UserDto> userMono = Mono.defer(() -> userRepository.findById("42"));

Flux<String> statusFlux = Flux.just("CREATED", "PAID", "SHIPPED")
    .map(String::toLowerCase);
```

`Mono.defer(...)` keeps creation lazy per subscriber, while `Flux` naturally models multiple values flowing over time.

Interview Trap

"`Mono.just(serviceCall())` is lazy because `Mono` is lazy."

That is wrong. `Mono` is lazy, but the argument to `just(...)` is evaluated immediately.

Quick Revision Notes

- `Mono` = 0..1, `Flux` = 0..N.
- Nothing happens until subscription.
- Assembly time and execution time are different.
- Trap to remember = `Mono.just(blockingCall())` runs the call immediately.

3. Event Loop and Netty Runtime

What It Changes in the Request Flow

This is where WebFlux becomes operationally different from Spring MVC.

Traditional MVC usually says:

- one request gets one worker thread
- that thread stays occupied until response completion

WebFlux with Netty says:

- a small set of event loop threads handle many requests
- when a request waits on non-blocking I/O, the thread goes back to serving other work

Simple Analogy

Imagine an airport check-in system:

- Blocking model: one staff member is assigned to one passenger and stands idle while that passenger searches for documents.
- Event loop model: one staff member keeps moving between many passengers, only interacting when each passenger is ready for the next step.

The staff member stays busy doing useful work instead of waiting.

Core Runtime Pieces

- `Netty` is the non-blocking networking engine.
- `Channel` represents a network connection.
- `EventLoop` is a thread that handles events for many channels.
- A channel is usually pinned to one event loop, which reduces coordination overhead.

How It Works Internally

1. Netty accepts the incoming HTTP connection.
2. The connection is assigned to an event loop.
3. That event loop processes read and write events for many channels.
4. Spring WebFlux handler returns `Mono` or `Flux`.
5. If the chain uses non-blocking downstream calls, the event loop is not parked waiting.
6. When readiness or completion events happen, the pipeline resumes and the response is written.

What This Means for Throughput

The main gain is not that one request becomes magically faster.

The gain is:

- fewer threads
- lower context switching
- lower memory pressure from thread stacks
- better behavior under large numbers of concurrent waiting requests

Why It Is Better Than Normal Flow

- Thread count stays smaller under load.
- Idle waiting does not pin worker threads.
- Streaming and connection-heavy workloads behave better.
- Event-driven networking scales well for APIs and gateways.

What Senior Interviewers Want to Hear

WebFlux works best when the whole call path is non-blocking. If you block the event loop, you destroy the advantage of the model.

Backfires That Might Occur

- Blocking JDBC call on the Netty event loop
- Large CPU-heavy JSON transformation on event loop thread
- File I/O or third-party SDK calls that block
- Assuming Netty alone fixes bad application design

When these happen, the event loop stalls and all channels assigned to that loop suffer.

Anti-Patterns

- Calling legacy blocking clients directly from the reactive chain without isolation
- Treating event loop threads as normal worker threads
- Running expensive crypto, compression, or report generation on the event loop
- Ignoring thread names during debugging and missing event loop starvation

Interview Punchline

Netty gives WebFlux its runtime advantage by replacing thread-per-request waiting with event-driven I/O on a small number of event loop threads.

Code Sample (Java)

```
Mono<String> nonBlockingFlow = webClient.get()
    .uri("/inventory/42")
    .retrieve()
    .bodyToMono(String.class);
```

```
Mono<String> isolatedBlockingFlow = Mono.fromCallable(() ->
legacyClient.fetch("42"))
    .subscribeOn(Schedulers.boundedElastic());
```

The first path stays non-blocking on the event loop. The second isolates a blocking call away from the event loop so it does not stall other requests.

Interview Trap

"Netty means any code inside WebFlux is safe because the runtime is non-blocking."

That is wrong. Netty is non-blocking, but your application code can still block the event loop.

Quick Revision Notes

- Event loop = small set of threads multiplex many connections.
- Benefit = fewer waiting threads, better concurrency density.
- Blocking on event loop hurts many requests, not just one.
- Trap to remember = non-blocking runtime does not make blocking code safe.

---

4. Spring WebFlux vs Spring MVC

First Clarification

Interviewers often say "WebFlux vs Spring Boot," but that comparison is slightly wrong.

- Spring Boot is the application bootstrap and auto-configuration layer.
- Spring MVC and Spring WebFlux are two different web programming models you can run with Spring Boot.

The real comparison is:

- Spring Boot + Spring MVC
- Spring Boot + Spring WebFlux

What Changes in the Request Flow

Spring MVC Flow

1. Servlet container assigns a request thread.
2. Controller executes on that thread.
3. Blocking calls keep the thread occupied.
4. Response returns when all work finishes.

Spring WebFlux Flow

1. Netty event loop receives request.
2. Controller or handler returns a publisher.
3. Non-blocking calls register continuation instead of parking a thread.
4. Response is produced from signals over time.

Simple Analogy

- Spring MVC is like assigning one clerk to each customer until the whole form is finished.
- WebFlux is like a token system where a small team keeps handling the next ready customer step.

Where WebFlux Clearly Wins

- High-concurrency APIs
- API gateways and edge services
- Streaming endpoints
- Chatty service-to-service calls with non-blocking clients
- Systems using reactive databases or messaging

Where MVC Is Still Better

- CRUD applications with blocking JDBC
- Simpler teams and simpler debugging needs
- CPU-bound processing
- Low to moderate concurrency systems where servlet scaling is already enough

Advantage of WebFlux Over Normal Flow

- Better thread utilization
- Better fit for streaming and real-time data flow
- Better resilience under large concurrent waits
- Easier end-to-end reactive composition with WebClient, R2DBC, reactive Kafka, and SSE

Cost of Choosing WebFlux

- Higher conceptual complexity
- Harder debugging when the team is inexperienced
- Easy to accidentally mix blocking and non-blocking code
- More operator semantics to understand deeply

Annotation Model vs Functional Endpoints

WebFlux supports both:

- Annotation style with `@RestController`
- Functional style with `RouterFunction` and `HandlerFunction`

Functional endpoints are sometimes preferred in fully reactive services because they make the pipeline more explicit, but annotation style is still common and interview-safe.

Backfires That Might Occur

- Migrating to WebFlux without replacing blocking repositories and clients
- Measuring only average latency and missing thread starvation under load
- Forcing reactive style onto a team that cannot support the debugging cost
- Believing WebFlux is automatically the best choice for every microservice

Anti-Patterns

- Using WebFlux just because it looks modern
- Running JDBC, JPA, or blocking SDKs in the request chain without scheduler isolation
- Mixing imperative transaction assumptions into reactive flow
- Overusing reactive style for trivial synchronous logic

Interview Punchline

Choose WebFlux when the bottleneck is concurrent I/O wait and you can keep the stack non-blocking. Choose MVC when the workload is simple, blocking, or team clarity matters more than concurrency density.

Code Sample (Java)

```
@RestController
class MvcUserController {
    @GetMapping("/mvc/users/{id}")
    UserDto getUser(@PathVariable String id) {
        return userService.findBlocking(id);
    }
}

@RestController
class WebFluxUserController {
    @GetMapping("/webflux/users/{id}")
    Mono<UserDto> getUser(@PathVariable String id) {
        return userService.findReactive(id);
    }
}
```

The important difference is not syntax alone. MVC expects the value now; WebFlux accepts a publisher and completes the response later.

Interview Trap

"Spring WebFlux is the replacement for Spring Boot."

That is wrong. Spring Boot is the framework setup layer. WebFlux and MVC are two different web stacks that can both run on Spring Boot.

Quick Revision Notes

- Compare WebFlux to Spring MVC, not to Spring Boot.
- WebFlux wins when concurrency + I/O wait dominate.
- MVC stays better for straightforward blocking applications.
- Trap to remember = modern does not mean universally better.

---

5. Project Reactor Lifecycle

What It Changes in the Request Flow

This topic explains what truly happens after your controller returns `Mono` or `Flux`.

In WebFlux, the request is not just "call method and get result." It is a signal lifecycle.

The Lifecycle in One Line

Assembly -> Subscription -> Demand -> Data -> Completion or Error -> Cleanup

Simple Analogy

Think of building and running a factory conveyor system:

- Assembly: you design the conveyor layout.
- Subscription: you switch the power on.
- Request: you decide how many boxes should come through now.
- onNext: boxes move one by one.
- onComplete: no more boxes are coming.
- onError: the belt jammed.
- cancel: stop the belt because the customer no longer needs the boxes.

Signal Order That Matters in Interviews

1. `onSubscribe`
2. `request(n)`
3. zero or more `onNext`
4. one terminal signal: `onComplete` or `onError`

Or cancellation can happen before natural completion.

How It Works in a WebFlux Request

1. Controller builds a pipeline.
2. Spring subscribes to it to serve the response.
3. The subscription is created.
4. Demand is requested.
5. Data signals flow through operators.
6. The response encoder writes values.
7. Terminal signal closes the response.
8. Cancellation can happen if the client disconnects or operators like `take` stop early.

Why This Lifecycle Matters

- Explains laziness
- Explains backpressure
- Explains why side effects may happen more than once on multiple subscriptions
- Explains why cleanup logic belongs in lifecycle-aware hooks

Important Hooks to Understand Early

- `doOnSubscribe`
- `doOnRequest`
- `doOnNext`
- `doOnError`
- `doOnComplete`
- `doOnCancel`
- `doFinally`

These are observation hooks, not primary business logic tools.

Small Example

```
Mono<String> pipeline = Mono.just("webflux")
    .doOnSubscribe(sub -> System.out.println("subscribed"))
    .map(String::toUpperCase)
    .doOnNext(value -> System.out.println("value = " + value))
    .doOnSuccess(value -> System.out.println("completed"));
```

Nothing is printed until subscription happens.

Why It Is Better Than Normal Flow

- The runtime has explicit lifecycle signals.
- Demand can be coordinated.
- Cancellation is first-class.
- Cleanup can be attached declaratively.

Traditional imperative code often hides these concerns inside thread blocking and try-finally blocks.

Backfires That Might Occur

- Duplicate side effects if the same cold publisher is subscribed to multiple times
- Misusing `doOnNext` for business mutations
- Forgetting that cancellation is normal and must be handled
- Not understanding terminal signals and accidentally swallowing failures

Anti-Patterns

- Putting core business transformations inside `doOn...` hooks
- Assuming a pipeline executes immediately after it is declared
- Ignoring cancellation and cleanup for long-running streams
- Sharing a cold publisher across multiple subscribers without understanding replay or caching behavior

Interview Punchline

Project Reactor is not just a fluent API. It is a signal protocol with a lifecycle, and understanding that lifecycle is the difference between writing reactive syntax and writing correct reactive systems.

Code Sample (Java)

```
Flux<Integer> pipeline = Flux.range(1, 5)
    .doOnSubscribe(sub -> System.out.println("subscribed"))
    .doOnRequest(n -> System.out.println("requested = " + n))
    .doOnNext(value -> System.out.println("next = " + value))
    .take(2)
    .doOnCancel(() -> System.out.println("cancelled"))
    .doFinally(signal -> System.out.println("final signal = " + signal));

pipeline.subscribe();
```

This shows the lifecycle as observable signals: subscribe, request, next values, cancellation from `take(2)`, and final cleanup.

Interview Trap

"`doOnNext` is a good place for core business logic because it runs for every item."

That is wrong. `doOn...` hooks are for observation and side effects, not the main transformation path.

Quick Revision Notes

- Lifecycle = assembly -> subscription -> demand -> signals -> terminal event.

- Cancellation is normal, especially in streaming or short-circuit operators.
- Multiple subscriptions can repeat side effects on cold publishers.
- Trap to remember = `doOn...` is not the primary place for business logic.

Batch 1 — The End-to-End Request Story

Use this as your interview narration when asked, "What really happens in WebFlux?"

1. A request arrives on Netty's event loop instead of being tied to a servlet worker thread for its whole lifetime.
2. The controller returns `Mono` or `Flux`, which is a publisher, not the final value.
3. Spring subscribes to that publisher to drive the response.
4. If downstream I/O is non-blocking, the thread is not parked while waiting.
5. Signals flow through Reactor operators until completion, error, or cancellation.
6. The main benefit is not one-request speed. The benefit is better concurrency handling when many requests spend time waiting.

Batch 1 — Connecting the Dots: One Real Request Story

Take this controller:

```
@GetMapping("/orders/{id}")
Mono<OrderView> getOrder(@PathVariable String id) {
    Mono<Customer> customerMono = customerClient.getCustomer(id);
    Mono<Payment> paymentMono = paymentClient.getPayment(id);
    Mono<Order> orderMono = orderRepository.findById(id);

    return Mono.zip(customerMono, paymentMono, orderMono)
        .map(tuple -> new OrderView(tuple.getT1(), tuple.getT2(),
tuple.getT3()));
}
```

Now the exact story is:

Step 1: Request reaches Netty

- The HTTP request reaches Reactor Netty.
- Netty assigns that connection's channel to one `EventLoop`.
- An `EventLoop` is basically one thread plus a task queue plus readiness handling for many sockets.
- There are many event loops in an `EventLoopGroup`, not just one global loop.
- One channel is usually handled by one event loop for its lifetime, which reduces locking.

Step 2: Spring invokes the controller

- Spring WebFlux calls `getOrder(id)`.
- This method runs now, on the request processing thread.
- Inside it, the pipeline is assembled.

Step 3: Assembly happens here

Assembly means: building the recipe, not cooking the food yet.

At this point:

- `customerMono` is created
- `paymentMono` is created
- `orderMono` is created
- `Mono.zip(...).map(...)` is created

Usually, if `customerClient`, `paymentClient`, and `orderRepository` are truly reactive, no HTTP call and no DB query has actually run yet.

You have only built a publisher graph.

Step 4: Controller returns the `Mono<OrderView>`

- The controller returns a `Mono<OrderView>` to Spring.
- That `Mono` is not the JSON response.
- It is a publisher that says, "When subscribed, I know how to eventually produce one `OrderView`."

Step 5: Spring subscribes

This is the missing dot for most people.

You do not subscribe manually in controller code. Spring does it for you.

Why?

Because Spring is the HTTP engine managing the response. It needs the values from your publisher to write the response body, so it subscribes on your behalf.

Subscription is the trigger that starts execution.

Step 6: Execution starts only now

After Spring subscribes:

- `zip` subscribes to `customerMono`
- `zip` subscribes to `paymentMono`
- `zip` subscribes to `orderMono`

That upstream subscription causes the real work to start:

- WebClient sends the first outbound HTTP call

- WebClient sends the second outbound HTTP call
- R2DBC sends the DB query

This is execution time.

Step 7: While waiting, no request thread is parked uselessly

- Netty event loop initiates non-blocking I/O work.
- The runtime registers interest in completion/readiness.
- The thread is free to process other ready events.

This is the heart of WebFlux.

The system is not doing less work. It is wasting fewer threads while waiting.

Step 8: Results come back as signals

When each external service or database response is ready:

- corresponding `Mono` emits `onNext(data)`
- then emits completion

`Mono.zip(...)` waits until all three Monos each produce one value.

Then:

- `map(...)` runs
- `OrderView` is created
- final `Mono<OrderView>` emits one value

Step 9: Spring writes plain JSON to the client

This is another key missing dot.

The frontend does not receive a `Mono` object.

The reactive type is only on the server side.

What Spring does after subscription produces a value:

1. receives the emitted `OrderView`
2. uses Jackson message writers to serialize it
3. converts it into JSON bytes
4. writes those bytes to the HTTP response
5. completes the response

So the browser or frontend client sees plain JSON because HTTP always carries bytes, not Java reactive types.

What if the controller returns `Flux<T>`?

Depends on content type.

- If you collect it into one JSON array, the client may receive a normal JSON array.
- If you use streaming media types like SSE or NDJSON, the client may receive chunks progressively.

So `Flux` can behave either like one full response or a true streamed response depending on how you expose it.

Why `Flux` is not just "list of Monos"

This is an important conceptual correction.

`Flux<T>` means one publisher that can emit many `T` values over time.

It is not the same thing as `List<Mono<T>>`.

Why not?

- `Flux<T>` has one subscription and one signal stream
- `Flux<T>` supports backpressure across the sequence
- `Flux<T>` can be infinite
- `Flux<T>` can stream items one by one
- `Mono<List<T>>` waits and emits one item: the whole list
- `List<Mono<T>>` is just many separate async containers with no single stream contract

Use this memory trick:

- `Mono<List<User>>` = one truck arrives carrying all users together
- `Flux<User>` = users arrive one by one on a conveyor belt

Do we have many event loops?

Yes.

Think of it like this:

- One `EventLoop` ~= one thread handling many channels
- Many `EventLoop`s form a group
- Connections are distributed across them

So the model is not:

- one event loop for the whole app
- one thread per request

The real model is:

- small pool of event loop threads
- each loop handles many connections
- each connection usually sticks to one loop

If one loop gets blocked, the channels attached to that loop suffer. The whole app may still run because other loops exist, but that blocked loop becomes a hotspot.

Final Mental Model

- Assembly = controller builds the pipeline now
- Subscription = Spring says "start producing data"
- Execution = external calls and DB query really happen now
- Signal flow = results travel back through operators
- Serialization = Spring turns final emitted object into JSON bytes
- Response = client receives normal JSON, not `Mono` or `Flux`

Exact Trigger Point: When Does Spring Actually Subscribe?

The precise trigger is: after your controller returns the `Mono` or `Flux`, Spring moves into response handling mode and subscribes because it now needs actual data to write into the HTTP response.

Very roughly, the internal flow is:

1. `DispatcherHandler` routes the request
2. `RequestMappingHandlerAdapter` invokes your controller
3. your controller returns `Mono<T>` or `Flux<T>`
4. Spring creates a `HandlerResult`
5. a result handler such as `ResponseBodyResultHandler` takes over
6. while writing the body, Spring subscribes to the publisher

So the trigger point is not:

- at application startup
- when the method is defined
- when the `Mono` variable is created

The trigger point is:

- after the controller returns
- when Spring is about to produce the HTTP response body

That is why your publisher stays lazy until the web framework needs the actual bytes.

Timing Analysis: Service 1 Takes 5 Seconds, Service 2 Takes 1 Second

Assume this code:

```
@GetMapping("/aggregate/{id}")
Mono<AggregateView> aggregate(@PathVariable String id) {
    Mono<Service1Response> service1 = service1Client.fetch(id);
    Mono<Service2Response> service2 = service2Client.fetch(id);
    Mono<Entity> db = repository.findById(id);
```

```
    return Mono.zip(service1, service2, db)
        .map(tuple -> new AggregateView(tuple.getT1(), tuple.getT2(),
tuple.getT3()));
}
```

Assume:

- service 1 takes 5 seconds
- service 2 takes 1 second
- DB takes 200 ms

Timeline

t = 0 ms

- Request arrives.
- Netty accepts it on one event loop.
- Spring invokes the controller.
- Assembly happens.
- `service1`, `service2`, `db`, and `zip(...).map(...)` are created.
- Controller returns `Mono<AggregateView>`.

No real remote work has happened yet if the sources are lazy and reactive.

t = 1 to 2 ms

- Spring's response handling layer subscribes.
- `zip` subscribes upstream to all three sources.
- WebClient sends outbound call to service 1.
- WebClient sends outbound call to service 2.
- R2DBC sends DB query.

This is the real execution start.

t = 200 ms

- DB response arrives.
- DB publisher emits its result.
- `zip` stores that value internally because it still needs service 1 and service 2.
- Nothing can be emitted downstream yet.

t = 1000 ms

- Service 2 response arrives.
- Service 2 publisher emits its result.
- `zip` stores it too.

- Still no final response yet, because service 1 has not finished.

t = 1000 ms to 5000 ms

This is where your main confusion usually lives.

What happens now?

- The current request is logically waiting for service 1.
- No thread is sitting blocked just to wait for that 5-second response.
- The event loop thread has already moved on to other ready work.

That other work can include:

- handling new incoming HTTP requests
- processing other sockets that became readable or writable
- finishing response writes for unrelated requests
- processing timer tasks and queued callbacks

So WebFlux does not mean "request is continuously running for 5 seconds."

It means "request state exists, but threads are reused for other ready work while this request waits on external I/O."

t = 5000 ms

- Service 1 response arrives.
- Service 1 publisher emits its result.
- Now `zip` finally has all three required values.
- `map(...)` runs and creates `AggregateView`.
- Final `Mono<AggregateView>` emits one value.
- Spring serializes it to JSON and writes the HTTP response.

What Exactly Does the Event Loop Do While Waiting?

It does not spin on your request.

It does not sit in a Java `Thread.sleep(...)`-like wait.

Instead:

- it registers interest in socket readiness
- it processes whichever channel becomes ready next
- it handles queued tasks
- it returns to the selector/event polling mechanism when nothing is ready

So when service 1 is still slow, the event loop can handle request 2, request 3, request 4, and other response events in the meantime.

What Is "the Next" Work Chosen By?

Mostly not by business priority.

It is usually driven by:

- which socket became ready for read/write
- which tasks are queued on that event loop
- scheduled timer tasks becoming due

For one channel, Netty preserves ordering by processing that channel's events serially on its assigned event loop.

Across many channels, the loop keeps taking ready work as it appears.

So the mental model is:

- not business-priority scheduling
- not one request fully to completion before the next request
- not random chaos either

It is event readiness plus queued tasks on a small set of loop threads.

If Service 2 Finishes Early, Does It Help?

Yes, but only partially.

It helps because:

- its result is already available
- no extra wait remains for that dependency

But if you use `zip`, the overall request still waits for the slowest dependency needed for the final combination.

So in this case:

- service 2 finishes early at 1 second
- DB finishes at 200 ms
- final response still waits for service 1 at 5 seconds

This is why aggregation latency is often dominated by the slowest required call.

Important Interview Observation

WebFlux improves concurrency and thread efficiency during the 5-second wait.

It does not remove the 5-second dependency latency itself.

If you want lower end-to-end latency, you need techniques like:

- timeout

- fallback
- partial response strategy
- cache
- parallel fan-out
- hedging or request racing in special cases

One-Line Memory Trick

Slow service means slow response time for that request, but not a blocked thread for that whole duration.

Do We Come Back to the Same Thread After 5 Seconds?

Usually, there is partial affinity, but do not think of it as "the same Java stack frame was paused and resumed on the same thread."

That is the wrong model.

The better model is:

- the request state is represented by reactive objects and subscribers
- network channels are attached to event loop threads
- when I/O becomes ready, the framework emits a signal into the subscriber chain

So what comes back is not "my sleeping method on the same thread."

What comes back is:

- an I/O readiness event
- decoded response bytes
- a reactive signal like `onNext` or `onComplete`

What Affinity Exists?

There are two important kinds of affinity to understand.

1. Channel-to-event-loop affinity

For Netty, a channel is usually assigned to one event loop for its lifetime.

That means:

- incoming server connection A is generally handled by one server event loop
- outbound client connection B is generally handled by one client event loop

This gives ordering and reduces locking.

2. Reactive operator thread affinity

By default, Reactor operators run on whatever thread delivers the signal.

So if a `WebClient` response arrives on a Reactor Netty client event loop thread, the downstream operator chain may continue on that thread unless you switch execution using `publishOn(...)` or `subscribeOn(...)`.

So the answer is:

- sometimes the continuation may run on the same event loop thread associated with that channel
- but you should not depend on "same thread always resumes everything"
- once schedulers are introduced, thread hops are normal

Then How Does the System Know the Job Is Done?

Because the reactive chain is already wired with subscribers and signal handlers.

You do not see an explicit callback because Reactor operators are themselves callback composition.

Under the hood, something like this conceptually exists:

1. Spring subscribes to your final publisher
2. `zip` subscribes to each source publisher
3. each source publisher registers downstream subscribers
4. Netty registers interest in network events
5. when bytes arrive, Netty notifies the relevant pipeline/handler
6. Reactor Netty turns that event into reactive signals
7. those signals invoke `onNext`, `onComplete`, or `onError` on the operator chain

So the callback is real. It is just hidden inside the framework.

Where Is the Hidden Callback in `zip`?

`Mono.zip(...)` creates internal subscribers, one for each source.

Conceptually it behaves like this:

- subscribe to source 1
- subscribe to source 2
- subscribe to source 3
- when source 1 emits, store value 1
- when source 2 emits, store value 2
- when source 3 emits, store value 3
- when all required values are present, invoke the combinator path and emit downstream

So `zip` does not poll.

It reacts.

It sits as a coordinator with internal state.

Think of It Like This

Imagine three couriers delivering parts to one assembly desk:

- courier A brings customer data
- courier B brings payment data
- courier C brings DB data

`zip` is the desk clerk.

The clerk does not repeatedly ask, "Are you here yet? Are you here yet?"

Instead:

- each courier arrives when ready
- the clerk stores each part
- once all three are present, the clerk assembles the package and passes it onward

That is what `zip` is doing with signals.

In Your 5-Second Example, What Exactly Happens?

Let us say service 1 uses client channel C1 and service 2 uses client channel C2.

At `t = 0`:

- Spring subscribes
- `zip` subscribes to service1/source1 and service2/source2 and DB/source3
- outbound requests are sent

At `t = 1 sec` service 2 completes:

- bytes arrive on client channel C2
- C2's event loop processes that read event
- Reactor Netty decodes the HTTP response
- source2 emits `onNext(response2)`
- `zip`'s internal subscriber for source2 stores that value
- `zip` still waits because source1 is missing

At `t = 5 sec` service 1 completes:

- bytes arrive on client channel C1
- C1's event loop processes that read event
- Reactor Netty decodes the HTTP response
- source1 emits `onNext(response1)`
- `zip`'s internal subscriber for source1 stores that value
- now `zip` has all values, so it emits downstream
- Spring's response writer receives the final object and writes JSON to the server response channel

So nothing needed to "wake up a sleeping thread."

The arriving network event triggered the next signal.

Important Correction

Reactive flow is not thread resumption in the old blocking sense.

Reactive flow is event arrival causing signal propagation through a pre-wired subscriber chain.

That is why you do not see manual callbacks in application code even though callback-style mechanics are absolutely happening underneath.

Batch 1 — Why `Mono.just(blockingCall())` Is Dangerous

This is one of the highest-value WebFlux interview traps.

The Exact Core Idea

Yes, the value is emitted later at subscription time.

But the dangerous part is that the blocking work already happened earlier.

When you write:

```
Mono<String> mono = Mono.just(blockingCall());
```

Java evaluates `blockingCall()` first, then passes its result into `Mono.just(...)`.

So the real sequence is:

1. run `blockingCall()` now
2. wait for it to finish now
3. create `Mono.just(result)` with the already computed value
4. later, on subscription, emit that already available value

So the emission is lazy, but the expensive work was not lazy.

Why This Is Dangerous in WebFlux

In WebFlux, assembly often happens on the request-processing path, commonly on a Netty event loop thread.

So if `blockingCall()` takes 5 seconds:

- that thread is blocked for 5 seconds during assembly
- the reactive pipeline is not even fully returned yet
- Spring cannot subscribe yet because controller execution itself is stuck
- if that thread is an event loop thread, other channels on that loop get delayed too

That means you destroyed the main WebFlux benefit before the reactive chain even started.

The Actual Damage

`Mono.just(blockingCall())` causes all of these problems:

1. It blocks the current thread immediately.
2. It breaks laziness for the expensive work.
3. If `blockingCall()` throws, the exception happens during assembly, not as a normal reactive source signal.
4. If nobody subscribes, or downstream cancels, the blocking call has already happened anyway.
5. It may run on the wrong thread, including a Netty event loop.

Why "Emits Later" Does Not Save You

Because by the time emission happens, the costly part is already over.

Think of it like this:

- `Mono.just(blockingCall())` = buy the food right now, then later hand someone a receipt
- `Mono.fromCallable(() -> blockingCall())` = hand them a coupon that actually buys the food only when redeemed

In the first case, the waiting already happened.

Small Request Timeline

```
@GetMapping("/demo")
Mono<String> demo() {
    return Mono.just(blockingCall());
}
```

Timeline:

1. request enters controller
2. `blockingCall()` runs immediately
3. current thread waits 5 seconds
4. `Mono.just(result)` is finally created
5. controller returns
6. Spring subscribes
7. result emits quickly because it was already computed

So yes, subscription-time emission is later, but the 5-second blocking already happened in the controller thread.

Safer Alternative for Blocking Code

```
Mono<String> mono = Mono.fromCallable(() -> blockingCall())
    .subscribeOn(Schedulers.boundedElastic());
```

Why this is better:

- `blockingCall()` is deferred until subscription
- it becomes part of reactive execution
- it runs on `boundedElastic`, not on the event loop
- exceptions can flow through the reactive chain

Most Important Interview Line

`Mono.just(x)` is lazy only for emitting `x`, not for computing `x`.

Batch 1 — Side-by-Side Timeline: `just` vs `fromCallable` vs `WebClient`

Use the same imaginary 5-second dependency for all three.

Case 1: `Mono.just(blockingCall())`

```
Mono<String> mono = Mono.just(blockingCall());
```

Assembly Phase

- `blockingCall()` runs immediately
- current thread blocks for 5 seconds
- only after it finishes does `Mono.just(result)` get created

Subscription Phase

- Spring subscribes after the controller returns
- value is already available
- `Mono` emits almost immediately

Thread Impact

- current request thread was blocked during assembly
- if this happened on an event loop, that loop was harmed
- no real reactive waiting benefit was preserved

Net Effect

- response still takes about 5 seconds
- concurrency benefit is mostly lost

Case 2: `Mono.fromCallable(() -> blockingCall()).subscribeOn(Schedulers.boundedElastic())`

```
Mono<String> mono = Mono.fromCallable(() -> blockingCall())
    .subscribeOn(Schedulers.boundedElastic());
```

Assembly Phase

- only the recipe is created
- no blocking call runs yet

Subscription Phase

- Spring subscribes
- Reactor schedules the callable on `boundedElastic`
- a worker thread from that scheduler executes `blockingCall()`

While Waiting

- request/event-loop thread is free
- a bounded-elastic worker is blocked instead
- that is acceptable because this scheduler is meant for blocking work

After 5 Seconds

- callable returns result on the bounded-elastic worker
- `Mono` emits downstream
- Spring serializes and writes response

Thread Impact

- blocking still exists, but it is isolated away from the event loop
- WebFlux concurrency is preserved better than in `Mono.just(blockingCall())`

Net Effect

- response still takes about 5 seconds
- but the wrong thread was not blocked

Case 3: `WebClient.get(...).retrieve().bodyToMono(...)`

```
Mono<String> mono = webClient.get()
    .uri("/remote")
    .retrieve()
    .bodyToMono(String.class);
```

Assembly Phase

- request pipeline is built

- no remote body is available yet
- no thread is blocked waiting for the remote service

Subscription Phase

- Spring subscribes
- Reactor Netty initiates outbound non-blocking HTTP I/O

While Waiting

- no thread is blocked for 5 seconds just to wait for the response
- Netty event loop keeps handling other ready network events
- request state is waiting logically, not as a sleeping thread

After 5 Seconds

- remote socket becomes ready
- Netty processes the read event
- Reactor Netty decodes the response
- `Mono` emits downstream
- Spring writes JSON response

Thread Impact

- no dedicated thread was pinned for the entire wait
- this is the real non-blocking model WebFlux is designed for

Net Effect

- response still takes about 5 seconds if remote service is slow
- but thread efficiency stays high under concurrency

Comparison Table

Case	When Work Starts	Does Current Thread Block?	Where Does Work Run?
`Mono.just(blockingCall())`	During assembly	Yes	Current thread
`fromCallable(...).subscribeOn(boundedElastic())`	On subscription	Not on event loop	Bounded elastic worker

`WebClient.bodyToMono(...)`	On subscription	No waiting thread pinned	Netty non-blocking I/O + signal callbacks

Final Memory Trick

- `Mono.just(blockingCall())` = blocking now, emitting later
- `fromCallable(...).subscribeOn(boundedElastic())` = blocking later, but on the right kind of thread
- `WebClient.bodyToMono(...)` = non-blocking later, event-driven all the way

Batch 1 — Common Backfires Across All Five Topics

- Using WebFlux with blocking JDBC and expecting reactive gains
- Calling `block()` in request processing
- Manually calling `subscribe()` inside application flow
- Doing CPU-heavy work on event loop threads
- Treating `Mono` and `Flux` as plain containers instead of signal publishers
- Using reactive style where simple MVC would be easier and safer

---

Batch 1 — Interview Hot Questions

1. Is WebFlux always faster than Spring MVC?

No. WebFlux is usually better at concurrency under I/O wait. It is not automatically better for CPU-bound work or blocking stacks.

2. Why does WebFlux need Netty?

It does not strictly need only Netty, but Reactor Netty is the common runtime because it provides non-blocking event-loop-based networking.

3. What is the biggest mistake teams make with WebFlux?

Keeping blocking repositories and clients in the middle of a reactive chain and assuming the framework alone makes the app scalable.

4. Why is `Mono.just(blockingCall())` wrong?

Because the blocking call happens immediately at assembly time, not lazily at subscription time.

5. What is the real difference between `Mono` and `CompletableFuture`?

`Mono` is part of a richer reactive stream model with operators, terminal signals, cancellation semantics, and backpressure-aware composition with `Flux`.

6. Why does subscription matter so much?

Because the pipeline is lazy. Without subscription, the work usually never starts.

7. Why can blocking on an event loop be catastrophic?

Because one blocked event loop thread delays many channels attached to that loop, not just one request.

Batch 1 — Revision Notes

- One-line summary: WebFlux wins by making waiting cheap, not by making slow work magically fast.
- Three keywords: event loop, lazy signals, non-blocking I/O.
- One trap: using reactive controllers on top of a blocking stack.
- One memory trick: MVC holds a thread while waiting; WebFlux holds a callback path while waiting.
